\Simple Inheritance

Exercise 1: Ethiopian Government Structure

Hierarchy:

Federal Government
↑
Regional Government
↑
Woreda Administration

This perfectly represents **inheritance hierarchy**.

Instruction

Write a Java program to exercise **Inheritance**. The program should model the Ethiopian government hierarchy.

- Create a base class `Government` with a method to provide public services.
- Extend it with a `RegionalGovernment` class that manages regional affairs.
- Further extend it with a `WoredaAdministration` class that manages local services.
- In the demo program, create a `WoredaAdministration` object and demonstrate how it inherits and uses methods from all levels of the hierarchy.

Solution

```
class Government {
    void provideService() {
       System.out.println("Government provides public services.");
    }
}
class RegionalGovernment extends Government {
    void manageRegion() {
       System.out.println("Regional government manages regional
                                  affairs.");
    }
}
class WoredaAdministration extends RegionalGovernment {
    void manageLocalServices() {
       System.out.println("Woreda administration manages local
                                  services.");
    }
}
public class GovernmentDemo {
    public static void main(String[] args) {
       WoredaAdministration woreda = new WoredaAdministration();
       woreda.provideService();
       woreda.manageRegion();
       woreda.manageLocalServices();
    }
}
```

Exercise 2: Water Body

Introducing a parent class `WaterBody`. Both `River` and `Lake` will inherit from it, demonstrating how common attributes and behaviors can be shared across subclasses.

Instruction

Write a Java program to exercise **Inheritance and Subclass Behavior**. The program should represent Ethiopian rivers and lakes.

- Create a parent class **`WaterBody`** with attributes such as name and region, and a method to describe location.
- Create a subclass **`River`** with an additional attribute for length and a method to show flow details.
- Create a subclass **`Lake`** with an additional attribute for area and a method to show expansion details.
- In the demo program, create objects for Abay, Awash, Lake Tana, and Lake Ziway.
- Demonstrate how the parent method **`describe()`** works for all, and how subclass-specific methods (**`flow()`**, **`expand()`**) add specialized behavior

Solution

```
class WaterBody {
    String name;
    String region;
    WaterBody(String name, String region) {
       this.name = name;
       this.region = region;
    }
    void describe() {
       System.out.println(name + " is located in " + region + ".");
    }
}
class River extends WaterBody {
    double length; // in kilometers
    River(String name, double length, String region) {
       super(name, region); // call parent constructor
       this.length = length;
    }
    void flow() {
       System.out.println(name + " flows through " + region + " with
                                a length of " + length + " km.");
    }
}
class Lake extends WaterBody {
    double area; // in square kilometers

    Lake(String name, double area, String region) {
       super(name, region); // call parent constructor
       this.area = area;
    }
    void expand() {
       System.out.println(name + " covers an area of " + area + " km²
                                in " + region + ".");
    }
}
```

Demo Program

```
public class EthiopiaWaterDemo {
    public static void main(String[] args) {
       River abay = new River("Abay (Blue Nile)", 1450, "Amhara
                                    Region");
       River awash = new River("Awash", 1200, "Oromia Region");
       Lake tana = new Lake("Lake Tana", 3156, "Amhara Region");
       Lake ziway = new Lake("Lake Ziway", 440, "Oromia Region");

       // Using parent method
       abay.describe();
       awash.describe();
       tana.describe();
       ziway.describe();

       // Using subclass-specific methods
       abay.flow();
       awash.flow();
       tana.expand();
       ziway.expand();
    }
}
```

Expected Output

```
Abay (Blue Nile) is located in Amhara Region.
Awash is located in Oromia Region.
Lake Tana is located in Amhara Region.
Lake Ziway is located in Oromia Region.
Abay (Blue Nile) flows through Amhara Region with a length of 1450 km.
Awash flows through Oromia Region with a length of 1200 km.
Lake Tana covers an area of 3156 km² in Amhara Region.
Lake Ziway covers an area of 440 km² in Oromia Region.
```

Exercise 3: Water Body – Subclass invokes the parent class's method

Instruction

Write a Java program to exercise **Inheritance and Method Reuse**. The program should show how subclasses can invoke parent methods.

- Create a superclass `WaterBodyInEthiopia` with attributes name and region, and a method `displayBasicInfo()`.
- Create a subclass `River` with additional attributes length and destination, and a method `displayRiverInfo()` that calls `displayBasicInfo()` before printing river details.
- Create a subclass `Lake` with an additional attribute area, and a method `displayLakeInfo()` that calls `displayBasicInfo()` before printing lake details.
- In the main program, create objects for Abay, Awash, Lake Tana, and Lake Ziway.
- Demonstrate how each subclass reuses the parent method while adding its own details

Solution

Superclass: WaterBodyInEthiopia

This class contains properties common to **all Ethiopian water bodies**.

Common attributes:

- **name**
- **region**

These apply to **both rivers and lakes**.

```
class WaterBodyInEthiopia {
    String name;
    String region;
    void displayBasicInfo() {
       System.out.println("Name: " + name);
       System.out.println("Region: " + region);
    }
}
```

Subclass: River - A river **is a specialized type of water body. Additional attributes specific to rivers:**

- **length**
- **destination**

```
class River extends WaterBodyInEthiopia {
    double length;
    String destination;

    void displayRiverInfo() {
       displayBasicInfo();
       System.out.println("Length: " + length + " km");
       System.out.println("Flows to: " + destination);
       System.out.println();
    }
}
```

Subclass: Lake - A lake **is another type of water body. Additional attribute:**

- **area**

```
class Lake extends WaterBodyInEthiopia {
    double area;

    void displayLakeInfo() {
       displayBasicInfo();
       System.out.println("Area: " + area + " km²");
       System.out.println();
    }
}
```

Main Program (Creating Objects)

```
public class Main {
    public static void main(String[] args) {
       River abay = new River();
       abay.name = "Abay River";
       abay.region = "Amhara";
       abay.length = 1450;
       abay.destination = "Sudan";

       River awash = new River();
       awash.name = "Awash River";
       awash.region = "Oromia";
       awash.length = 1200;
       awash.destination = "Lake Abbe";

       Lake tana = new Lake();
       tana.name = "Lake Tana";
       tana.region = "Amhara";
       tana.area = 3050;

       Lake ziway = new Lake();
       ziway.name = "Lake Ziway";
       ziway.region = "Oromia";
       ziway.area = 485;

       abay.displayRiverInfo();
       awash.displayRiverInfo();
       tana.displayLakeInfo();
       ziway.displayLakeInfo();
    }
}
```

Question:

What is the expected output?

Exercise 4: CoffeeCermenoy Rounds

In the traditional **Ethiopian Coffee Ceremony**, coffee is served in three stages:

1. **Abol** – the first and strongest round
2. **Tona** – the second round
3. **Baraka** – the final blessing round

All are **coffee rounds**, but each **behaves differently**. This perfectly illustrates method overriding.

Instruction

Write a Java program to exercise **Polymorphism**. The program should model the three rounds of the Ethiopian coffee ceremony.

- Create a parent class `CoffeeRound` with a method `serve()`.
- Create subclasses `Abol`, `Tona`, and `Baraka` that override or extend behavior to represent each round.
- Each subclass should provide its own description of the round.
- In the demo program, create objects for all three rounds and demonstrate how the same `serve()` method behaves differently depending on the subclass

Solution

```
public class CoffeeRound {
    public void serve() {
       System.out.println("Serving coffee round.");
    }
}
// Children inherit serve() but do not override it
class Abol extends CoffeeRound {
    void describe() {
       System.out.println("Abol – the first and strongest coffee.");
    }
}
class Tona extends CoffeeRound {
    void describe() {
       System.out.println("Tona – the second round of coffee.");
    }
}
class Baraka extends CoffeeRound {
    void describe() {
       System.out.println("Baraka – the blessing round of coffee.");
    }
}
public class CoffeeInheritanceDemo {
    public static void main(String[] args) {
       Abol abol = new Abol();
       abol.serve();  // inherited from CoffeeRound
       abol.describe();   // specific to Abol

       Tona tona = new Tona();
       tona.serve();      // inherited
       tona.describe();   // specific to Tona

       Baraka baraka = new Baraka();
       baraka.serve();    // inherited
       baraka.describe(); // specific to Baraka
    }
}
```

Inheritance – Constructor and super keyword

Exercise 5: Fasting Traditions

Instruction

Write a Java program to exercise **Inheritance**. The program should manage Ethiopian fasting traditions.

- Create a base class `Tradition` with attributes such as name and duration.
- Create a subclass `Fast` that extends `Tradition`.
- Add a method to display fasting details.
- In the main class, store multiple fasting traditions in a list and print their details.
- Demonstrate how inheritance allows `Fast` to reuse and extend the behavior of `Tradition`

Solution

```
class Tradition {
    protected String name;
    protected int duration;
    public Tradition(String name, int duration) {
       this.name = name;
       this.duration = duration;
    }
    public void display() {
       System.out.println(name + " - " + duration + " days"); }
    }
class Fast extends Tradition {
    public Fast(String name, int duration) {
       super(name, duration);
    }
}
import java.util.*;
public class FastingDemo {
    public static void main(String[] args) {
       List fasts = new ArrayList<>();
       fasts.add(new Fast("Ramadan", 30));
       fasts.add(new Fast("Dhul Hija", 10));
       fasts.add(new Fast("Abiy Tsom", 55));
       fasts.add(new Fast("Gena", 30));
       for (Fast f : fasts) {
          f.display();
       }
    }
}
```

Exercise 6: Musical Instruments

Instruction

Write a Java program to exercise **Inheritance**. The program should manage Ethiopian musical instruments.

- Create a base class `Item` with attributes such as name and category.
- Create a subclass `Instrument` that extends `Item`.
- Add a method to display instrument details.
- In the main class, store multiple instruments in a list and print their details.
- Demonstrate how inheritance allows `Instrument` to reuse and extend the behavior of `Item`

Solution

```
class Item {
    protected String name;
    protected String category;
    public Item(String name, String category) {
       this.name = name;
       this.category = category;
    }
    public void display() {
       System.out.println(name + " - Category: " + category);
    }
}
class Instrument extends Item {
    public Instrument(String name, String category) {
       super(name, category);
    }
}
import java.util.*;
public class InstrumentDemo {
    public static void main(String[] args) {
       List instruments = new ArrayList<>();
       instruments.add(new Instrument("Masenqo", "String"));
       instruments.add(new Instrument("Washint", "Wind"));
       instruments.add(new Instrument("Kebero",Percussion"));
       instruments.add(new Instrument("Begena", "String"));
       for (Instrument i : instruments) {
          i.display();
       }
    }
}
```

Exercise 7: Musical Styles (ቅኝት)

Instruction

Write a Java program to exercise *Inheritance*. The program should manage Ethiopian ቅኝት (poetic/musical styles).

- Create a base class `Style` with attributes `name` and `origin`.
- Create a subclass `Kignit` that extends `Style`.
- Add a method to display style details.
- In the main class, store multiple styles in a list and print them.

Solution

```
class Style {
    protected String name;
    protected String origin;
    public Style(String name, String origin) {
       this.name = name;
       this.origin = origin;
    }
    public void display() {
       System.out.println(name + " - Origin: " + origin);
    }
}
class Kignit extends Style {
    public Kignit(String name, String origin) {
       super(name, origin);
    }
}
import java.util.*;
public class KignitDemo {
    public static void main(String[] args) {
       List styles = new ArrayList<>();
       styles.add(new Kignit("Ambasel", "Wollo"));
       styles.add(new Kignit("Zelesegna", "Shewa"));
       styles.add(new Kignit("Bati", "Wollo"));
       styles.add(new Kignit("Anchihoye", "Gondar"));
       for(Kignit k : styles) {
          k.display();
       }
    }
}
```

Exercise 8: Traditional Food

Instruction

Write a Java program to exercise *Inheritance*. The program should manage Ethiopian traditional food details.

- Create a base class `Food` with attributes `name` and `prepTime`.
- Create a subclass `TraditionalFood` that extends `Food`.
- Add a method to display food details.
- In the main class, store multiple foods in a list and print them.

Solution

```
class Food {
    protected String name;
    protected int prepTime;
    public Food(String name, int prepTime) {
       this.name = name;
       this.prepTime = prepTime;
    }
    public void display() {
       System.out.println(name + " - " + prepTime + " minutes");
    }
}
class TraditionalFood extends Food {
    public TraditionalFood(String name, int prepTime) {
       super(name, prepTime);
    }
}
import java.util.*;
public class FoodDemo {
    public static void main(String[] args) {
       List foods = new ArrayList<>();
       foods.add(new TraditionalFood("Injera", 60));
       foods.add(new TraditionalFood("Shiro", 30));
       foods.add(new TraditionalFood("Doro Wat", 90));
       for (TraditionalFood f : foods) {
          f.display();
       }
    }
}
```

Exercise 9: Coffee Ceremony

Instruction

Write a Java program to exercise *Inheritance*. The program should manage Ethiopian coffee ceremony details.

- Create a base class `Ceremony` with attributes `name` and `cups`.
- Create a subclass `CoffeeCeremony` that extends `Ceremony`.
- Add a method to display ceremony details.
- In the main class, store multiple ceremonies in a list and print them.

Solution

```
class Ceremony {
    protected String name;
    protected int cups;
    public Ceremony(String name, int cups) {
       this.name = name;
       this.cups = cups;
    }
    public void display() {
       System.out.println(name + " Ceremony - " + cups + " cups");
    }
}
class CoffeeCeremony extends Ceremony {
    public CoffeeCeremony(String name, int cups) {
       super(name, cups);
    }
}
import java.util.*;
public class CoffeeDemo {
    public static void main(String[] args) {
       List ceremonies = new ArrayList<>();
       ceremonies.add(new CoffeeCeremony("Abol", 3));
       ceremonies.add(new CoffeeCeremony("Tona", 3));
       ceremonies.add(new CoffeeCeremony("Baraka", 3));
       for (CoffeeCeremony c : ceremonies) {
          c.display();
       }
    }
}
```

Exercise 10: Holidays and Festivals (including National Holidays)

Instruction

Write a Java program to exercise *Inheritance*. The program should manage Ethiopian holidays and festivals.

- Create a base class `Event` with attributes `name` and `duration`.
- Create subclasses `Festival` and `NationalHoliday` that extend `Event`.
- Add a method to display event details.
- In the main class, store multiple events in a list and print them.

Solution

```
class Event {
    protected String name;
    protected int duration;
    public Event(String name, int duration) {
       this.name = name;
       this.duration = duration;
    }
    public void display() {
       System.out.println(name + " - " + duration + " days");
    }
}
class Festival extends Event {
    public Festival(String name, int duration) {
       super(name, duration);
    }
}
class NationalHoliday extends Event {
    public NationalHoliday(String name, int duration) {
       super(name, duration);
    }
}
import java.util.*;
public class EventDemo {
    public static void main(String[] args) {
       List events = new ArrayList<>();
       events.add(new Festival("Timket", 3));
       events.add(new Festival("Meskel", 1));
       events.add(new Festival("Eid", 3));
       events.add(new NationalHoliday("Adwa Victory Day", 1));
       events.add(new NationalHoliday("Patriots' Victory Day", 1));
       for (Event e : events) {
          e.display();
       }
    }
}
```

Inheritance – Method Overriding

Exercise 11: Ethiopian Music Scales (ቅኝት)

Ethiopian music has different **scales (ቅኝቶች)**:

- Tizita
- Bati
- Ambassel
- Anchihoye

Each scale **produces music differently**, but they all share the same concept: **play music**.

Instruction

Write a Java program to exercise **Method Overriding and Polymorphism**. The program should model Ethiopian music scales.

- Create a base class `MusicScale` with a method `play()`.
- Create subclasses `Tizita`, `Bati`, and `Ambassel` that override the `play()` method to produce different outputs.
- In the demo program, store multiple scales in a list or array of type `MusicScale`

Solution

```
class MusicScale {
     void play() {
        System.out.println("Playing Ethiopian music scale.");
     }
}
class Tizita extends MusicScale {
     void play() {
        System.out.println("Playing Tizita scale.");
     }
}
class Bati extends MusicScale {
     void play() {
        System.out.println("Playing Bati scale.");
     }
}
class Ambassel extends MusicScale {
     void play() {
        System.out.println("Playing Ambassel scale.");
     }
}
public class MusicDemo {
     public static void main(String[] args) {
        MusicScale m1 = new Tizita();
        MusicScale m2 = new Bati();
        MusicScale m3 = new Ambassel();
        m1.play();
        m2.play();
        m3.play();
     }
}
```

Exercise 11: Cultural and Natural Heritages – Prevent Overriding

Instruction

Write a Java program to exercise **Inheritance, Method Overriding, and Dynamic Method Dispatch**. The program should manage Ethiopian cultural and natural heritages.

- Create a base class `Heritage` with attributes such as name and location, and a method `showInfo()`.
- Add a `static` method in `Heritage` that defines heritage in general terms. Demonstrate that it cannot be overridden.
- Create a subclass `CulturalHeritage` that extends `Heritage` and adds an attribute for a related festival. Override the `showInfo()` method to display cultural details.
- Add a `final` method in `CulturalHeritage` that provides a cultural note, ensuring subclasses cannot override it.
- Create a subclass `NaturalHeritage` that extends `Heritage` and adds an attribute for a natural feature. Override the `showInfo()` method to display natural details.
- Create further subclasses of `CulturalHeritage` (e.g., `LalibelaChurches`, `HararJugol`) that add unique features.
- In the main class, store mixed objects (`CulturalHeritage`, `NaturalHeritage`, and their subclasses) in an **array of type `Heritage`**.
- Demonstrate **dynamic method dispatch** by calling `showInfo()` on each object in the array.
- Use the **`instanceof` keyword and type casting** to safely access subclass-specific methods.
- Show how the `final` and `static` methods behave in this hierarchy.

Solution

```
class Heritage {
    protected String name;
    protected String location;
    public Heritage(String name, String location) {
       this.name = name;
       this.location = location;
    }
    public String getName() { return name; }
    public String getLocation() { return location; }
    public void showInfo() {
       System.out.println(name + " is located in " + location + ".");
    }
    public static void heritageDefinition() {
       System.out.println("Heritage refers to cultural or natural
                           treasures passed down through generations.");
    }
}

class CulturalHeritage extends Heritage {
    private String festival;
    CulturalHeritage(String name, String location, String festival) {
       super(name, location);
       this.festival = festival;
    }
```

```
    @Override
    public void showInfo() {
       System.out.println(getName() + " in " + getLocation() +
                " is linked to the festival: " + festival + ".");
    }
    public final void culturalNote() {
       System.out.println("Cultural heritage is preserved across
                            generations.");
    }
}
// Subclasses of CulturalHeritage
class LalibelaChurches extends CulturalHeritage {
    LalibelaChurches() {
       super("Rock-Hewn Churches of Lalibela", "Amhara Region", "Genna
       Festival");
    }
    public void uniqueFeature() {
       System.out.println("Known for monolithic rock-hewn churches
       carved in the 12th century.");
    }
}
class HararJugol extends CulturalHeritage {
    HararJugol() {
       super("Harar Jugol (Old Walled City)", "Harari Region", "Islamic
       Traditions");
    }
    public void uniqueFeature() {
       System.out.println("Famous for its ancient walls and being a
       center of Islamic culture.");
    }
}
public class HeritageDemo {
    public static void main(String[] args) {
       // Array of mixed Heritage objects
       Heritage[] heritages = {
          new LalibelaChurches(),
          new HararJugol(),
          new NaturalHeritage("Simien Mountains National Park", "Amhara
          Region", "Dramatic landscapes"),
          new NaturalHeritage("Lake Tana", "Amhara Region", "Source of
          the Blue Nile")
       };
       // Loop through array to demonstrate dynamic dispatch
       for (Heritage h : heritages) {
          h.showInfo();  // resolved at runtime

          // Type casting with instanceof
          if (h instanceof LalibelaChurches) {
             ((LalibelaChurches) h).uniqueFeature();
          } else if (h instanceof HararJugol) {
             ((HararJugol) h).uniqueFeature();
          } else if (h instanceof NaturalHeritage) {
             System.out.println("This is a natural heritage site.");
          }
```

```
            // If it's cultural, call final method
            if (h instanceof CulturalHeritage) {
               ((CulturalHeritage) h).culturalNote();
            }
            System.out.println(); // spacing
         }
         // Static method demonstration
         Heritage.heritageDefinition();
      }
}
```